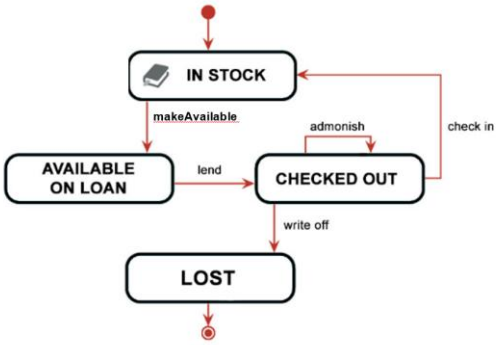
Instituto Superior de Engenharia
Politécnico de Coimbra

Testes e Qualidade de Software
Mestrado em Engenharia Informática
Laboratory Class 3

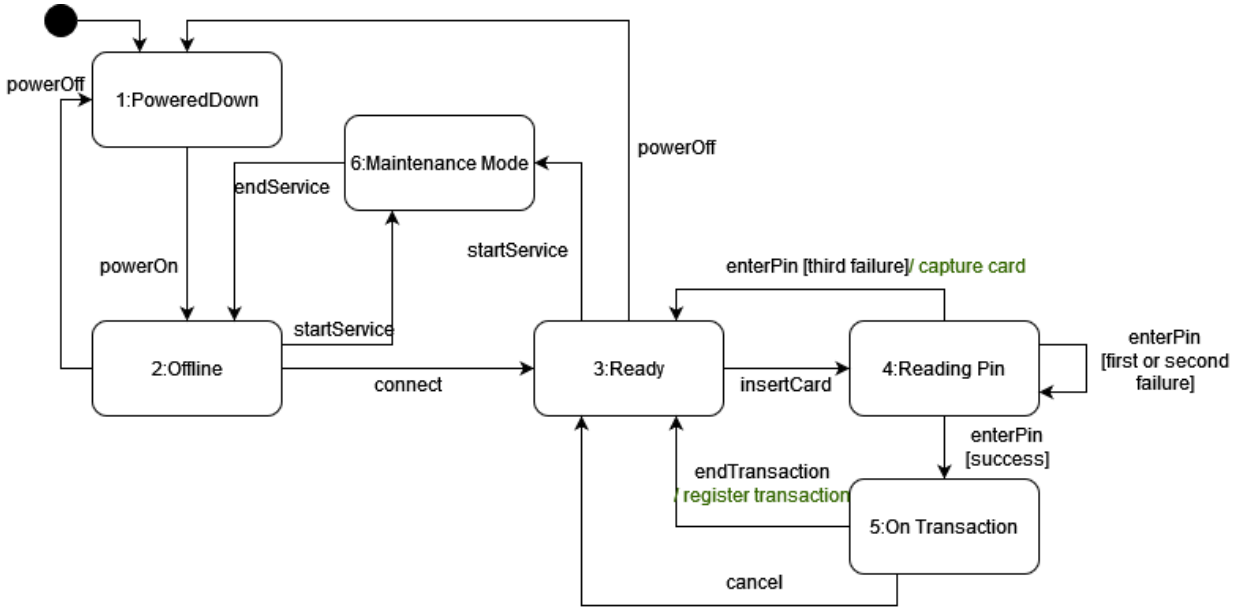
1 – Consider a `nextDate` method that receives a date as parameter and returns the next date. Design a test suite using the *decision table* approach. *(tip: consider what kind of actions would make it easy to represent the logic on a decision table)*

2 – Consider the following state diagram describing the status of a library book.



a) Design a test set satisfying state coverage.
b) Design a test set satisfying 1-switch coverage.
c) Design a test set for invalid transition testing

3- Consider the following state diagram describing the operation of an ATM machine.



You have been given the responsibility of designing a test suite for this system. Testing is expected to be conducted incrementally in three different stages of increasing demand.
On each one of these stages, a new test suite is created by adding new test cases to the test suite developed on the previous stage (rather than rebuilding a new test case from scratch).
To ensure the rationale behind test design is made clear for management, auditing and certification purposes, each test suite should satisfy a clearly defined state testing criteria, that subsumes the testing criteria of the previous stage.

a) Design the test suite for the first stage of testing. Identify explicitly the testing criteria you are satisfying, the test requirements, and the test suite that satisfies them. An example of a possible test case description for this system is as follows:

Test Number	Description	Input Sequence	Expected Outcome
1	User enters wrong pin three times; card should be captured.	powerOn/ connect/ insertCard/ enterPin[fail]/ enterPin[fail]/ enterPin[fail]	Card captured Final state is *Ready*

b) Likewise for the second stage test suite.
c) Likewise for the third stage test suite

3-Consider a system that for performing reservation of seats. These seats are classified as first class or second class, and the clients are also categorized as being priority or regular clients.

The system should handle reservations in the following way:

- Reservation request by regular clients will always be assigned a second class seat, if one is available. If no second class seat is available, an unoccupied first class seat will be assigned instead (this is called <u>overbooking</u>). If neither first nor second class seats are available, then the reservation fails.

- Reservation requests by priority clients will always be assigned an unoccupied first class seat. If no such seat exists, an overbooked first class seat is assigned instead (and the reservation of the overbooking regular client canceled) . If all first class seats are already taken by other priority clients, then they will be assigned a free second class seat instead. If no free second class seat exists, the reservation of a regular client will be canceled and his seat will be assigned to the priority client instead. Finally, if no reservation of a regular client exists to be canceled, then the reservation request will fail.

An implementation of this reservation system can be found in the class Reservation available in the jar file `reservations.jar` . Import this file into your project. The following interface is provided via the `Reservations` class of the package `reservations`:

Reservation(int firstClass, int secondClass) – constructor allows specification of the number of first and second class seats.

reserveRegular() , reservePriority() – requests reservation of seat for regular/priority client. Returns class of assigned seat (1,2) or 0 if no reservation could be made.

reserveRegular(int) , reservePriority(int) - request reservation of multiple regular/priority seats. Returns the number of seats effectively reserved (regardless of class of seats).

getPriorityReservations (int) , getRegularReservations (int) - returns the number of currently reserved seats for priority/regular clients.

getFreeFirstClass () , getFreeSecondClass () - returns the number of free first/second class seats.

cancelRegular () , cancelPriority () - cancels a reservation of a regular/priority client.

getOverbooked () – returns the number of overbooked regular clients.

lock() – Finishes the reservation process, not allowing any further changes.

Here is an example program using the class:

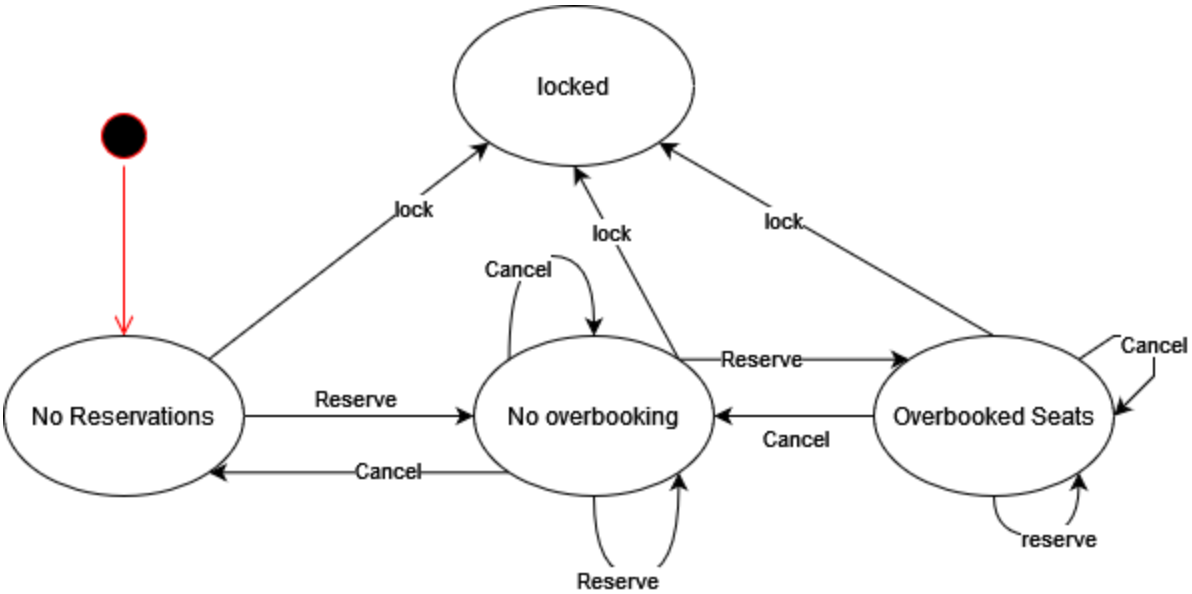
```
import reservations.Reservations;

public class Main {

	public static void main(String[] args) {
		Reservations r=new Reservations(10,25);
		int number=r.reserveRegular(30);
		System.out.println("Reserved "+number+" seats!");

	}
}
```

Your task is to design, implement, and run a test suit, using a black-box approach resorting to the techniques we have learned so far. Use JUnit to implement the tests.

a) Design a test suite using the space partitioning approach we studied in classes. Document your approach (what are the partitions, equivalence classes, coverage criteria, did you focus on inputs/outputs/internal state or variables, etc) and call the Professor to present and discuss your solution once you are done.

b) Design a test suite using the decision table approach we studied in classes. Document your approach (e.g. record the decision table) and call the Professor to present and discuss your solution once you are done.

c) Design a test suite using the state transition approach we studied in classes. Document your approach (what is the coverage criteria, etc) and call the Professor to present and discuss your solution once you are done. Consider the following state diagram for this purpose (guard conditions are not represented).


locked
lock
lock
lock
Cancel
Reserve
Cancel
No Reservations
Reserve
No overbooking
Overbooked Seats
Cancel
Cancel
reserve
Reserve